

作　　品　基于区块链的 AIGC 全生命周期版权管理系统

学　　校　天津大学

团队成员　马立原 胡登铖 栾博越 吕一然 韦人硕

时　　间　2024 年 12 月 8 日

目录

一、 目标问题与意义 ..... – 1 –
1. 项目背景 ..... – 1 –
2. 问题定义 ..... – 1 –
3. 相关工作与挑战 ..... – 2 –
二、 总体设计与方案 ..... – 3 –
1. 需求分析 ..... – 3 –
2. 系统架构 ..... – 4 –
3. 系统工作流程 ..... – 5 –
三、 方案实现 ..... – 6 –
1. 硬件资源 ..... – 6 –
2. 软件系统 ..... – 8 –
3. 算法实现 ..... – 8 –
四、 运行效果 ..... – 12 –
1. 实验设计 ..... – 12 –
2. 实验结果 ..... – 13 –
五、 特色与创新 ..... – 17 –
六、 未来展望 ..... – 17 –
七、 参考文献 ..... – 18 –
八、 附件 ..... – 19 –

一、 目标问题与意义

1. 项目背景

随着**用户生成内容**（UGC）与**人工智能生成内容**（AIGC）技术的迅速发展，内容生成的质量和复杂性显著提升，并逐渐应用于多媒体网络环境的各个领域。这些技术的广泛应用正推动内容生产方式的变革，为多媒体系统带来更高的创作效率和多样化的表现形式。根据 Precedence Research 的数据，全球 AIGC 市场规模预计将在 2030 年达到约 731.6 亿美元，展现出巨大的商业潜力。《中国 AI 数字商业展望 2021-2025》报告中指出，到 2025 年中国 AIGC 应用规模预计上升至 2070 亿元，2020-2025 年间的年均复合增长率高达 84.1%。然而，AIGC 的快速发展也伴随着严峻的挑战：在模型训练、内容生成及其应用等多个环节，传统的内容管理机制已难以适应分布式协作、实时追踪和版权保护的复杂需求，尤其在数据合规性与安全性方面存在突出问题。这些问题不仅限制了 AIGC 技术的进一步普及，也为其商业化应用带来了阻力。2023 年 7 月，国家网信办联合发改委、教育部、科技部、工信部、广电总局发布的《生成式人工智能服务管理暂行办法》要求生成内容不得侵害他人依法享有的知识产权。

区块链技术以其**去中心化**、**透明**、**防篡改**等特性，为解决 AIGC 面临的挑战提供了强有力的技术支撑。在模型训练阶段，区块链能够对训练数据的来源、合法性及使用权限进行存证，确保数据的合规性和可追溯性；在内容生成阶段，结合智能合约技术，可对模型调用记录及生成行为进行自动化监管，保障生成过程的透明性和可信度；在生成结果使用阶段，通过链上特征认证机制，可实现内容的唯一标识、版权归属确认和多版本的溯源管理，为内容创作者提供可靠的版权保护。

因此，区块链不仅可以覆盖 AIGC 的全生命周期管理，还通过去中心化架构，解决中心化系统中的信任问题，为 AIGC 的内容生成与协作提供完整、可靠的管理框架。本项目旨在基于区块链技术，构建一个高效、透明且可扩展的 AIGC 全生命周期管理系统，从模型训练到内容生成再到生成结果的使用，全面提供系统化的技术支持，推动 AIGC 技术的健康发展。

2. 问题定义

尽管 AIGC 技术已取得显著进展，但现有管理体系在多个关键环节存在显著不足，亟需解决。首先，在**模型训练阶段**，AIGC 模型高度依赖于大规模数据集，但数据来源的合法性审查和使用权限管理不到位，可能引发数据侵权或滥用的问题，进一步影响模型的可信性和合规性。此外，训练过程中样本数据缺乏数字水印等版权保护机制，导致数据来源与授权范围难以追溯，也为生成内容的可信性留下隐患。

在 AIGC 生成阶段，由于**缺乏模型调用记录和生成行为的透明化管理**，生成过程的行为监督不足，增加了生成内容被滥用的风险。尤其是在缺乏隐式水印的情况下，无法实现唯一标识与归属验证，难以有效管理版本迭代。此外，生成行为（如模型版本、输入参数及生成时间）未被绑定至具体生成内容，无法为后续的认证和溯源提供可靠依据。

在生成内容的管理阶段，现有机制**缺乏统一的认证标准和溯源体系**，难以对生成内容进行唯一标识、版权归属确认及版本追踪。这种不足使内容传播和二次创作过程中容易被篡改或未经授权使用，侵害创作者权益，也阻碍了多版本协作的高效迭代。

更为关键的是，目前尚无覆盖 **AIGC 全流程**的统一管理体系，各环节之间的信息记录和行为追踪无法形成闭环，难以满足 AIGC 分布式协作和多版本迭代的需求。针对上述问题，本项目提出了基于区块链的 AIGC 全生命周期综合管理框架，覆盖从训练到生成再到结果使用的各个环节。通过区块链的不可篡改性和透明性，结合隐式水印技术与智能合约，本项目设计了全方位的技术方案。在训练阶段，通过嵌入隐式数字水印，通过嵌入隐式数字水印并将关键源数据进行链上存证，确保数据来源可信、授权范围明确，并实现训练过程的全程可追溯。在生成阶段，利用区块链存储生成行为日志及水印验证信息，增强模型调用透明度与内容可信度，同时嵌入隐式水印，实现生成内容的唯一标识与版权确认。在结果使用阶段，通过基于区块链的溯源体系及多版本链式水印管理，构建高效的版本演化追踪机制，明确传播路径和版权归属，并实现侵权检测与收益分配的自动化管理。

通过构建覆盖训练、生成和使用的区块链管理闭环，本项目旨在为 AIGC 的分布式协作与全生命周期管理提供系统化技术支撑，全面提升 AIGC 在内容生成与管理中的可信性和效率，推动 AIGC 技术在数据合规、透明管理及版权保护中的广泛应用。

3. 相关工作与挑战

相关工作：在 AIGC 的版权保护与管理领域，学术界和工业界已经提出了一些解决方案，但仍存在不足之处，有待进一步优化和完善。

Liu 等人提出了 PoAIGC 框架，通过设计挑战协议（Challenge Protocol）检测内容抄袭与伪造，从而取消错误内容的注册。该方法结合了一种基于声誉的策略用于选择边缘服务提供商，提升了系统的灵活性和动态适应性。然而，PoAIGC 在 AIGC 内容认证和溯源设计上存在缺陷，未能覆盖从内容生成到使用的完整生命周期管理框架。此外，该方案未对其安全性进行正式分析，无法全面评估其抵御攻击或篡改的能力。

Lin 等人提出了一种基于区块链的语义通信框架，旨在解决分布式环境下的数据交换与信任构建问题。该框架通过验证机制确保数字内容的真实性，保证 AIGC 的可信性。然而，该方案倾向于提升生成内容的信任度，而在高效认证与内容溯源方面的能力较为薄弱，难以满足复杂版权管理场景的需求。

Wang 等人提出了一种结合区块链的 AIGC 服务系统，用于在元宇宙中建立分布式信任管理。该框架通过 IRM 方法提取跨环境的语义信息，并设计了基于智能合约的验证机制以确保随机结果的产生。此外，提出了基于 Stackelberg 博弈的内容缓存机制，以优化元宇宙参与者的收益。然而，该方法在 AIGC 服务的训练数据合规性与生成内容的全生命周期管理方面未能提供端到端的认证与版权保护框架。

其他相关研究也尝试采用前沿技术以提升 AIGC 内容的可信性。例如，零知识证明技术能够在保护隐私的同时验证实时生成内容的可信度。然而，仅仅依靠零知识证明等隐私保护方法仍然无法构建与 AIGC 实体之间的关联，无法覆盖内容生成与管理全生命周期。

本项目在解决 AIGC 全生命周期管理问题时面临以下**核心挑战**：

（1）**如何保障模型训练阶段的数据合规性与隐私安全。**AIGC 模型训练依赖于大规模数据集，但在数据来源的合法性验证、使用权限管理和隐私保护方面存在显著不足。数据来源不透明不仅带来数据侵权或滥用风险，还可能削弱了模型的可信性。此外，现有隐私保护机制难以与 AIGC 实体建立有效关联，无法有效标识训练数据的来源及授权范围，限制了溯源能力。

（2）**如何实现内容生成阶段的高效认证与使用记录。**AIGC 内容生成过程中，透明性和可信度管理机制薄弱，尤其是缺乏对模型调用记录和生成行为的系统性管理，导致内容滥用和伪造风险增加。与此同时，由于缺少版权可信编码和唯一标识，导致归属验证和版权保护难以实现。

（3）**如何构建生成结果使用阶段的全生命周期溯源体系。**AIGC 生成结果在传播与二次创作过程中，面临版权归属模糊、版本管理复杂及演化路径不可追踪等问题。现有机制缺乏统一的认证标准和溯源体系，难以实现内容的版权归属确认及版本迭代路径的精准追踪，限制了 AIGC 内容的规范化发展。

二、 总体设计与方案

1. 需求分析

本项目需要通过结合区块链技术和 AIGC 生成机制满足以下需求：

（1）在**模型训练阶段**，通过链上存证确保训练数据的来源合法性和授权范围的透明记录，利用隐私保护技术验证数据的合规性。此外，将训练数据与合法授权信息关联，构建数据来源溯源机制，确保生成内容源头可追溯。

（2）在**内容生成阶段**，需要生成内容提供唯一标识，通过结合区块链记录关键生成行为，确保生成过程的透明性和可追溯性。同时，将实体内容与版权信息编码关联，防止滥用或伪造，确保生成内容的合规性和可信性。

（3）在**生成内容使用与管理阶段**，需要通过多版本溯源机制记录生成内容的传播路径和演化关系，明确每次修改或二次创作的版权归属。基于隐式编码与区块链技术构建动态的授权管理机制，确保内容传播过程透明、规范。

2. 系统架构

系统由三个核心模块组成，**分别是水印模块、标识模块、编码模块和解析模块，**系统的架构如图 2-1 所示。

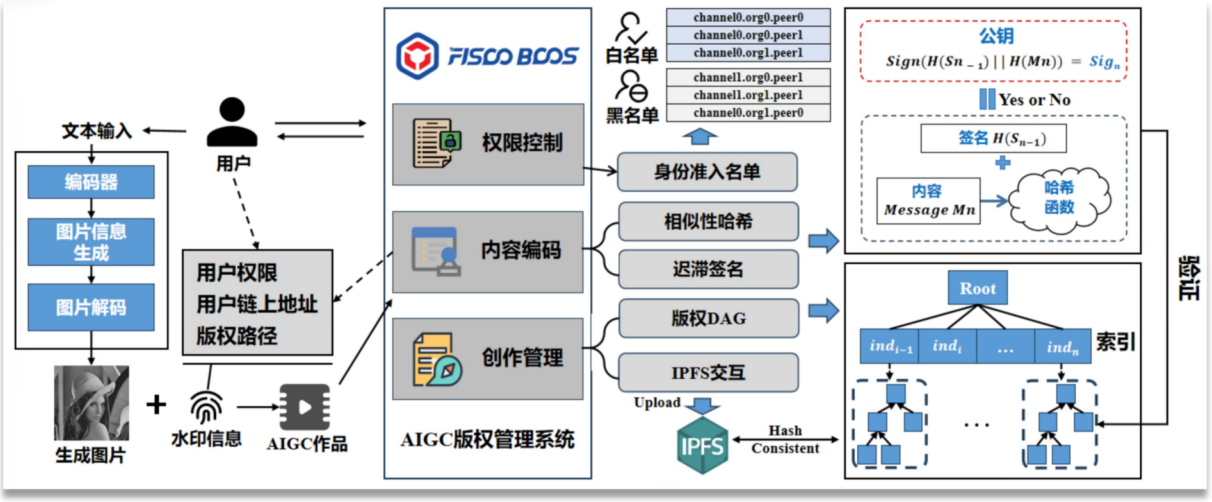


图 2-1 系统架构图

水印模块用于在生成阶段初期嵌入数字水印，确保每个生成内容都具备唯一标识。这种数字水印技术在内容生成时被嵌入，并且不可见，以便后续的版权认证、内容追溯和侵权检测。水印模块还为衍生内容提供了相应的水印嵌入功能，以保持内容演化过程中的版权链条完整性和可追溯性。

标识模块负责用户身份管理和访问控制，通过白名单机制实现严格的服务访问控制，确保系统的安全性和完整性。用户通过发送注册请求至身份管理模块进行身份注册，系统将注册请求转发至链上智能合约进行身份验证，并动态分配资源访问权限。该模块还负责用户身份唯一性验证及动态权限管理，为内容生成和数据访问建立信任基础。

编码模块用于管理生成内容的版权，定义最小版权单元（MCU），并通过滞后签名 DAG（HSD）创建 AIGC 实体与版权持有者之间的安全、可追溯的联系。编码模块记录生成内容的元数据、生成模型版本和输入参数，并通过链上存证确保内容的生成行为透明、合法。该模块还通过隐式水印为每个生成内容分配唯一标识，支持后续的版权认证与追溯，确保内容的真实性和版权管理。

解析模块负责生成内容的存储和管理，基于相似性哈希指针构建 DAG 数据结构，用于数据的高效检索和存储，并通过局部敏感哈希（LSH）实现快速检索。此外，解

析模块利用内容标识符（CID）在 IPFS 中进行确定性映射存储，确保生成内容的可追溯性和版本管理。解析模块还支持对生成内容的二次创作与管理，通过分布式存储系统链接原始内容和衍生内容，记录版权归属和创作路径，提供高效的溯源与版本管理。

3. 系统工作流程

本系统通过结合区块链技术与 AIGC 生成机制，实现从模型训练到内容生成以及生成内容版权管理的全流程覆盖。涵盖用户身份管理、数据存证、内容生成与认证、衍生内容管理与溯源、版权争议追踪与解决、以及授权收益分配等功能。系统工作流程分为**系统准备阶段**和**系统运行阶段**，如图 2-2 所示。

（1）系统准备阶段具体系统执行流程如下：

步骤 1-1：**用户身份注册与权限分配。**用户通过发送请求消息 *register(userID, userInfo)* 至身份管理模块进行身份注册。系统将注册请求转发至链上智能合约进行身份验证。AIGCCM 根据用户身份类型分配资源访问权限，并通过交易 *RegisterAIGC(userEnroll, userID, Certificate)* 记录用户身份及权限信息。该模块确保用户身份唯一性，并提供动态权限管理，为后续数据访问和内容生成建立信任基础。

步骤 1-2：**模型训练数据的合规性存证。**AIGCM 收集多模态数据（文本、图像、音频等）作为 AIGC 模型的训练数据，对每条数据的来源、授权范围和隐私保护状态生成元信息，并通过交易 *RegisterData(dataHash, userID, metadata)* 在区块链上存证。为提升数据质量，系统通过隐式水印对数据进行标识，确保数据来源可追溯。同时结合零知识证明技术，在数据验证时保护敏感信息的隐私。

（2）系统运行阶段包括内容生成与版权管理模块，具体操作步骤如下：

步骤 2-1：**内容生成请求与生成行为记录。**注册用户根据权限发送内容生成请求 *CreateAIGC(Msg)*，其中 *Msg* 包括内容的元数据、生成模型版本和输入参数。系统接收请求后，生成内容特征的 *SimHash* 值，并将其存储在区块链上。实际生成的内容通过分布式存储系统（如 IPFS）保存，系统同时记录调用模型的版本信息和生成时间，确保内容生成行为透明、合法。

步骤 2-2：**生成内容的唯一标识与认证。**系统为每个生成内容分配唯一标识，并通过链上存证明确生成内容的来源、生成时间和使用权限。用户可通过交易 *QueryAIGC(queryType, queryValue)* 查询生成内容的相关信息，包括生成行为、数据来源及模型调用历史，实现对生成内容真实性、合法性的验证。隐式水印嵌入机制确保生成内容具备唯一标识，支持后续的版权认证与追溯。

步骤 2-3：**生成行为的协作管理。**在多模型协同生成任务中，系统通过区块链记录所有模型调用过程及生成行为，包括模型版本、输入参数和输出结果。协作生成过程中，区块链为每个参与模型生成唯一记录节点，确保生成行为的透明性和可审计性。

步骤 3-1：**衍生内容的生成与管理。**用户在权限范围内对生成内容进行二次创作，通过交易 *CreateAIGC(Msg)* 更新衍生内容的链上存证信息。系统通过分布式存储系统链接原始内容和衍生内容，利用隐式水印为新内容分配唯一标识，并在区块链上记录衍生内容的版权归属及演化路径。

步骤 3-2：**内容的溯源与版本管理。**系统构建版权管理数据结构，记录每次内容演化的链式关联节点，形成演化链条。用户可通过交易 *VerifyAIGC(Hash)* 验证内容的版权归属及演化路径，系统通过链上签名确认内容的所有权和使用合法性。演化路径的记录和水印嵌入确保每个版本均可精准溯源。

步骤 3-3：**版权争议与违规处理。**在发生版权争议时，系统通过版权管理数据结构的历史记录追溯相关节点，并通过签名链验证内容的原创性。对于检测到的侵权内容，系统通过交易 *DeleteAIGC(Hash)* 将其标记为无效，同时触发智能合约限制内容传播。该模块结合侵权检测与链上存证，为版权纠纷提供快速解决方案。

步骤 3-4：**内容的授权与收益分配。**系统动态管理生成内容的授权范围，包括传播限制、二次创作许可及商业使用授权。用户可通过交易 *TransferAIGC(Hash, userID)* 转移内容所有权或修改授权信息。

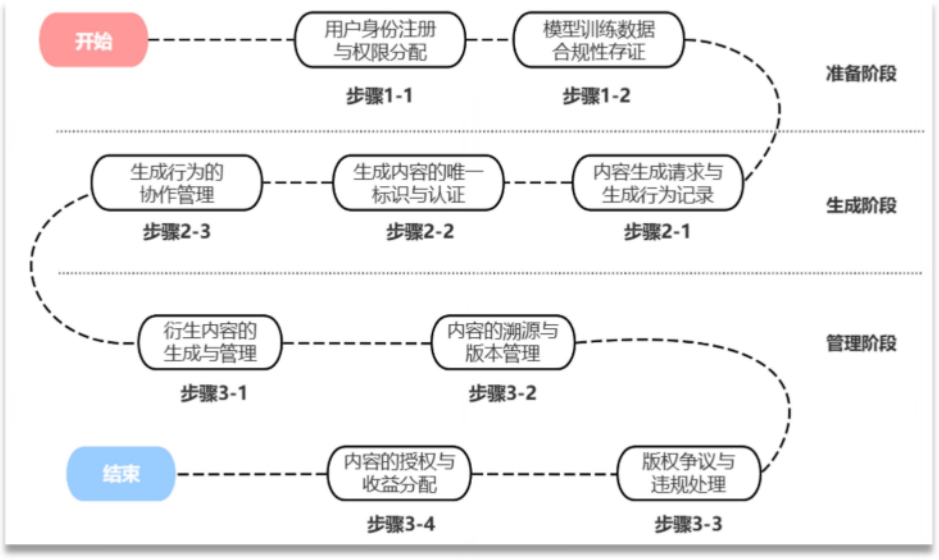


图 2-2 系统工作流程图

三、 方案实现

1. 硬件资源

CUDA 是由 NVIDIA 推出的用于通用并行计算的并行计算平台和编程模型，是一种新的操作 GPU 计算的硬件和软件架构，它提供了 GPU 编程的简易接口，CUDA 编程可以构建基于 GPU 计算的应用程序，利用 GPUs 的并行计算引擎来更加高效地解决

比较复杂的计算难题。它将 GPU 视作一个数据并行计算设备，而且无需把这些计算映射到图形 API。操作系统的多任务机制可以同时管理 CUDA 访问 GPU 和图形程序的运行库，其计算特性支持利用 CUDA 直观地编写 GPU 核心程序。CUDA 提供了一种类似于 C 语言的编程模型，开发人员可以使用 CUDA C/C++语言编写并行程序，并通过 CUDA 工具链将其编译成针对 NVIDIA GPU 的可执行文件。通过 CUDA，开发人员可以直接访问 GPU 的计算能力，实现高性能的并行计算，例如图形处理、科学计算、机器学习等领域的应用。GPU 服务器是一种配备了高性能图形处理单元（GPU）的服务器。相对于传统的服务器，GPU 服务器通过集成强大的 GPU 加速卡提供了额外的并行计算能力，可用于加速各种计算密集型任务，如图形处理、深度计算、数据分析等。华为云图形加速增强型 G6 服务器配套 NVIDIA Tesla T4 GPU 显卡，能够满足专业级的图形处理需求。系统整体部署如图 3-2 所示，硬件资源参数如表 3-1 所示。

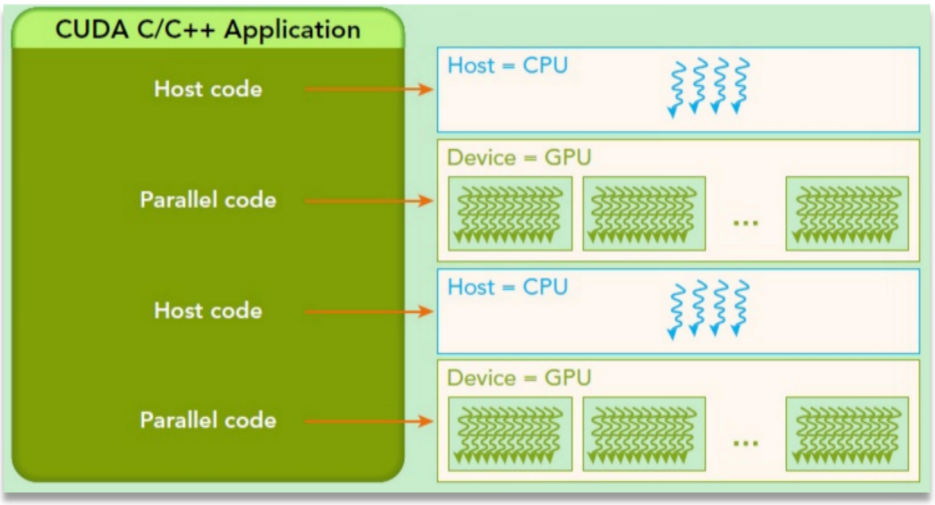


图 3-1 CUDA 程序执行流程图

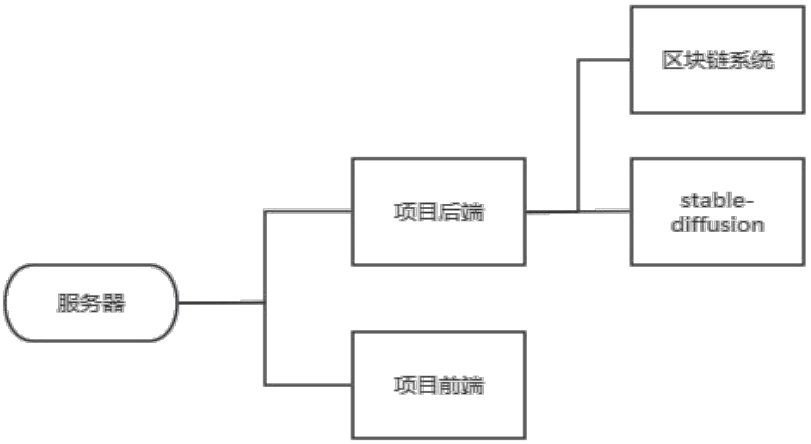


图 3-2 系统整体部署图

表 3-1 硬件资源参数

参数	数值
区块链	FISCO BICOS v2.0
节点信息	3 x HUAWEI CLOUD c6s.2xlarge.2
服务器带宽	15 (25) Gbit/s
内存	16 GiB
CPU	8v Intel(R) Xeon(R) Gold 6348 CPU @ 2.60GHz
GPU	4 * NVIDIA Tesla V100 / 4 * 16GiB
服务器位置	中国-北京、南非-约翰内斯堡、墨西哥-墨西哥城
代码量	7000 LOC Go/Solidity/JavaScript

2. 软件系统

系统软件环境包括:

(1) 系统环境：Ubuntu 18.04；主要编程语言：Go。

(2) 系统前端框架：Vue 框架；编程语言：JavaScript，TypeScript。

(3) 系统后端框架：Gin 框架；编程语言：Go，前后端通信架构：RESTful API。

(4) 区块链开发平台：FISCO-BCOS 平台；前端管理工具：WeBase。

(5) 智能合约编程语言：Go、Solidity。

(6) AIGC 模型：Stable Diffusion 模型；编程语言：Go。

3. 算法实现

(1) **系统初始化**

用户注册。用户通过发送注册请求消息 *Register(userID, userInfo)* 至身份管理模块完成注册。系统将接收到的 *userInfo* 转化为唯一地址后存储到链上，确保用户身份信息的真实性与唯一性。

数据实体-编码绑定。系统收集多模态数据 $D = \{d_1, d_2, ..., d_n\}$ 作为 AIGC 模型的训练集。对每条数据 d_i，系统计算其 *SimHash* 值，并通过接口 $RegisterData(sh_i, userID, m_i)$ 在链上存证，并记录数据的来源和授权范围，确保数据的合法性和可追溯性。同时，系统对每条数据 d_i 嵌入隐式水印 W_i，W_i 由 sh_i，*userID* 和时间戳 *t* 组成，并生成特征向量 w。以图片为例，系统对 w_i 进行离散小波变化，生成频域系数 C，并通过公式 $C' = C + \alpha w_i$ 进行水印嵌入，并通过逆变换重构为带隐式水印的图像 d_i'。

数据合规性验证。为在验证数据合规性并保护敏感信息，系统采用零知识证明技术保证数据隐私，提供者根据数据 d_i 计算 $P = H(d_i || R)$，其中 R 为随机数，用于生成

唯一的承诺值。验证者随机生成挑战e，提供者根据e和R计算水印验证信息$s = eH(d_j)$并返回给验证者。验证者根据$C = H(d_i || (s - eH(d_j)))$的一致性确保数据的合法性，同时保证用户隐私。

（2）版权保护

系统通过区块链的去中心化与防篡改特性，将生成内容的核心数据存储在链上，实现内容的唯一标识和版权登记。同时，智能合约中管理版权的 DAG 数据结构使得版权保护流程更加自动化，从内容创作到版本迭代，再到授权管理和收益分配，均可实时处理，减少人为干预带来的效率问题和信任成本。此外，系统引入白名单机制，通过智能合约严格控制用户对内容的访问权限和操作权限。这种权限管理不仅保护了内容的安全性，还保障了用户隐私，防止未经授权的访问或篡改行为。

1. 内容生成的存证与唯一标识

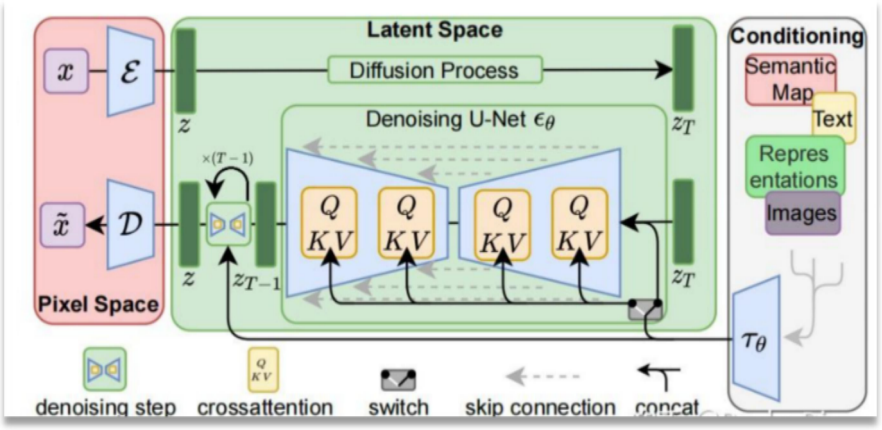


图 3-3 stable_diffusion 模型整体架构图

```go
func (dag *DAG) Insert(newNode *Node) {
    currentNode := dag.Root
    rootSimilarity := Similarity(currentNode.Hash, newNode.Hash)
    bestMatch, _ := dag.findBestMatch(currentNode, newNode.Hash, rootSimilarity)
    newSimilarity := Similarity(bestMatch.Hash, newNode.Hash)
    level := getIntervalNumber(newSimilarity)
    newNode.Prehash = bestMatch.Hash
    bestMatch.AddChild(level, newNode)
    dag.UpdateAffectedNodes(newNode)
}

func (dag *DAG) Delete(deleteHash string) bool {}
func (dag *DAG) UpdateAffectedNodes(node *Node) {}
func (dag *DAG) VerifySignature(path []Node) bool {
    for i := len(path) - 1; i > 0; i-- {
        if !dag.normalSignatureVerify(path[i]) {
            return false
        }
        if !dag.verifyHashAgreement(path[i], path[i-1]) {
            return false
        }
    }
    return true
}
```

图 3-4 信息加入区块链记录实现

在内容生成阶段，项目以 Stable Diffusion 为例完成生成任务，Stable Diffusion 是一种基于扩散模型的改进方法，通过引入隐向量空间来解决扩散模型中的速度瓶颈问题，特别适用于图像生成任务，模型架构如图 3-3 所示。在生成过程中，系统通过文本语义特征提取和随机高斯噪声作为隐空间特征的初始输入，U-Net 网络结合调度算法对隐空间特征进行动态优化，将文本语义注入隐空间，通过解码器将优化后的隐空间特征还原为高质量的像素图像。与此同时，在输出最终图像前，结合步骤 1-2 中的隐式水印编码算法，将相关版权信息写入生成图像中，为后续验证和溯源提供可靠依据。最终，由后端将生成的图片返回前端。

系统会为每项生成内容创建链上存证。具体而言，使用 SimHash 算法提取内容经过压缩所获得的矩阵特征值，生成固定长度的哈希标识值。这个标识值连同创作者信息、生成时间等信息形成元数据，赋予内容一个独特的数字身份。

链上记录确保了内容的唯一性与可追溯性。一旦数据写入区块链，无论是创作时间、创作者信息，还是内容特征，都将成为不可篡改的历史记录，为后续的版权追踪与争议解决提供坚实的技术基础。

2. 版权登记与所有权管理

在内容生成后，系统会自动通过智能合约完成版权登记。该过程生成一份版权凭证，将创作者与内容的数字标识绑定在一起，明确内容的所有权。完成绑定后，系统会调用交易 *CreateNode()*方法，自动将版权凭证封装为新的 Node 并记录到区块链系统之中，完成版权的链上校验与认证检查。

```
type Node struct {
    Hash      string             `json:"hash"`
    Prehash   string             `json:"prehash"`
    Matrix    [][]float64        `json:"matrix"`
    Style     string             `json:"style"`
    Make      string             `json:"make"`
    Record    string             `json:"record"`
    Owner     string             `json:"owner"`
    HSnMinus1 string             `json:"HSnMinus1"`
    HMn       string             `json:"HMn"`
    SIGN      []byte             `json:"SIGN"`
    Children  map[string][]*Node `json:"children"`
}
```

图 3-5　DAG 节点内容

如果创作者希望转移内容的所有权，系统提供了 *TransferOwnership(Hash)*方法。该方法通过哈希签名验证转移请求的真实性，并在完成转移后自动更新所有权记录，完成版权所有权的转移，确保变更过程透明、安全且不可篡改。

3. 内容溯源与版本管理

针对内容的版本管理和演化路径，系统设计了基于多 DAG（有向无环图）结构的数据存储机制。在 DAG 中，每个根节点表示某一类特定的作品集合，而子节点则依

次代表该作品的不同版本或衍生作品。每个结点均存储着对应类型的版权凭证合约，每次内容的二次创作或修改，都会在 DAG 中生成一个新的子节点存储新创作作品的凭证，并通过链上数据结构与原节点关联。

```go
func (dag *DAG) findBestMatch(currentNode *Node, newSimhash string, similarityValue float64) (*Node, float64) {
    bestMatch := currentNode
    bestSimilarity := similarityValue

    for _, children := range currentNode.Children {
        for _, child := range children {
            childSimilarity := Similarity(child.Hash, newSimhash)
            if childSimilarity > bestSimilarity {
                bestMatch, bestSimilarity = dag.findBestMatch(child, newSimhash, childSimilarity)
            }
        }
    }
    return bestMatch, bestSimilarity
}
func (dag *DAG) GetPath(queryNode *Node) []Node {
    var path []Node
    currentNode := dag.Root
    path = append(path, *currentNode)
    for currentNode.Hash != queryNode.Hash {
        bestMatch := new(Node)
        bestSimilarity := -math.MaxFloat64
        similarityValue := Similarity(currentNode.Hash, queryNode.Hash)
        level := getIntervalNumber(similarityValue)
        levelString := strconv.Itoa(level)
        if children, ok := currentNode.Children[levelString]; ok {
            for _, child := range children {
                childSimilarity := Similarity(child.Hash, queryNode.Hash)
                if childSimilarity >= bestSimilarity {
                    bestSimilarity = childSimilarity
                    bestMatch = child
                }
            }
        }
        if bestMatch == nil {
            return nil
        }
        path = append(path, *bestMatch)
        currentNode = bestMatch
    }

    return path
}
```

图 3-6 基于 DAG 版权溯源数据结构

DAG 能够保证内容的演化路径在链上清晰可见。用户可以通过溯源功能查询某一版本的版权归属、生成时间及与原内容的关系，确保每一阶段的创作过程可被验证与追踪。

4. 侵权检测与争议解决

为了保护版权免受侵害，系统利用 SimHash 算法快速检索内容的相似性，并对疑似侵权行为进行自动检测。在版权登记与版权合约凭证生成的过程中，系统会自动检测该新加入的合约的 Simhash 哈希标识与链上已有版权的哈希标识的相似度。如果检测到内容侵权，系统会触发智能合约的争议解决机制。通过比对链上记录的数据，系统能够验证侵权内容的来源及其与合法内容的相似程度。如果用户提起相似度仲裁，可以通过交易 *Querysimilarity(Hash)* 来人工申请重新执行上述流程进行检查。

```go
// Simhash 函数计算一组词元的Simhash值
func Simhash(tokens []Token) string {
    if len(tokens) == 0 {
        return EMPTY_HASH
    }

    jenkins := NewJenkins() // 假设这是创建Jenkins实例的函数

    var memo []int
    for _, token := range tokens {
        hash := toBinary(jenkins.Hash64(token.Text))
        array := make([]int, 64) // 假设hash是64位的二进制字符串
        for i, char := range hash {
            sign := -1
            if char == '1' {
                sign = 1
            }
            array[i] = sign * token.Weight
        }

        // 合并数组
        if memo == nil {
            memo = make([]int, 64)
            copy(memo, array)
        } else {
            for i := range memo {
                memo[i] += array[i]
            }
        }
    }

    // 降维
    var result []byte
    for _, v := range memo {
        if v > 0 {
            result = append(result, '1')
        } else {
            result = append(result, '0')
        }
    }

    return string(result)
}
```

图 3-7　Simhash 算法实现

对于确认侵权的内容，系统会通过智能合约将其标记为无效，并从有向无环图（DAG）上的节点之中删除该侵权的版权凭证合约，限制其在平台上的传播。同时，侵权争议的全过程，包括检测结果与处理决定，都会记录在链上，确保解决过程公开透明。

5. 授权与收益分配

内容的使用权限和收益分配由智能合约动态管理。用户可以通过合约设置传播限制、商业使用许可等授权范围。例如，当内容被授权用于二次创作或商业用途时，系统会通过链上记录追踪其使用情况。

四、 运行效果

1. 实验设计

如图 4-1 所示，本系统针对真实的文生图场景进行了详细实验测试和分析，分别部署了用户前端、硬件端和区块链端，涵盖**用户身份认证、训练数据合规性存证、内容生成和唯一标识、衍生内容协作与管理、版权争议处理**等核心环节，以及**查询、创作、修改、删除**等关键操作。



图 4-1 实验部署环境图

2. 实验结果

本项目旨在基于区块链技术，构建一个**高效**、**透明**且**可扩展**的 AIGC 全生命周期管理系统，实验核心目的即为**管理系统的高效成功搭建以及运行**，体现在用户身份注册与权限分配，内容生成请求与生成行为记录，生成内容的唯一标识与认证，生成行为的协作管理，衍生内容的生成与管理，内容的溯源与版本管理。

举例如下：系统开始运行并且搭建了网页为使用者提供**可视化界面**，如图 4-2,4-3 所示，前端可视页面中，用户进行身份注册，在前端传入生成图片描述，经过系统首先会反馈所生成内容。

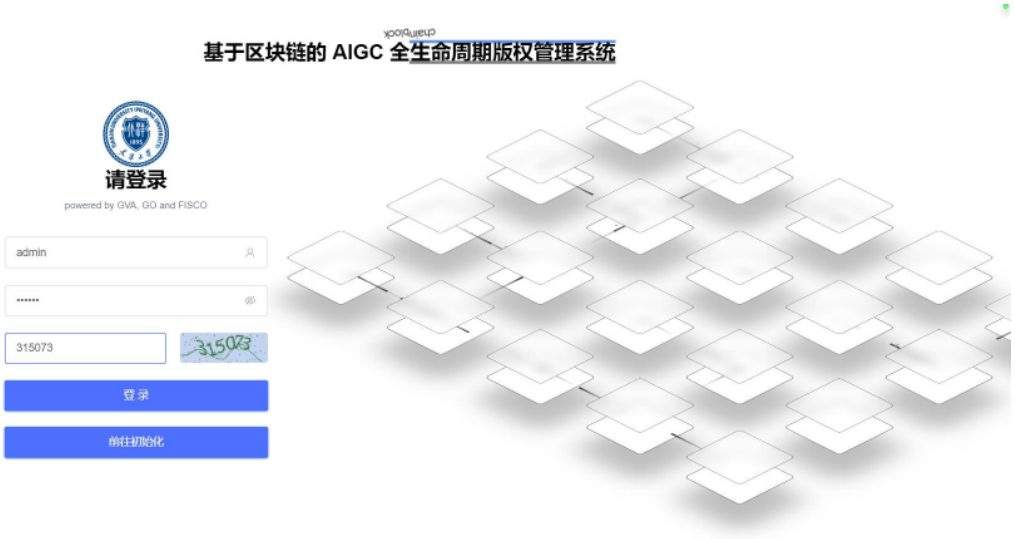


图 4-2 用户注册界面



图 4-3 内容生成界面

同时在项目后端，采用模块化与分层的思想实现数据库与链协同管理,如图 4-4。

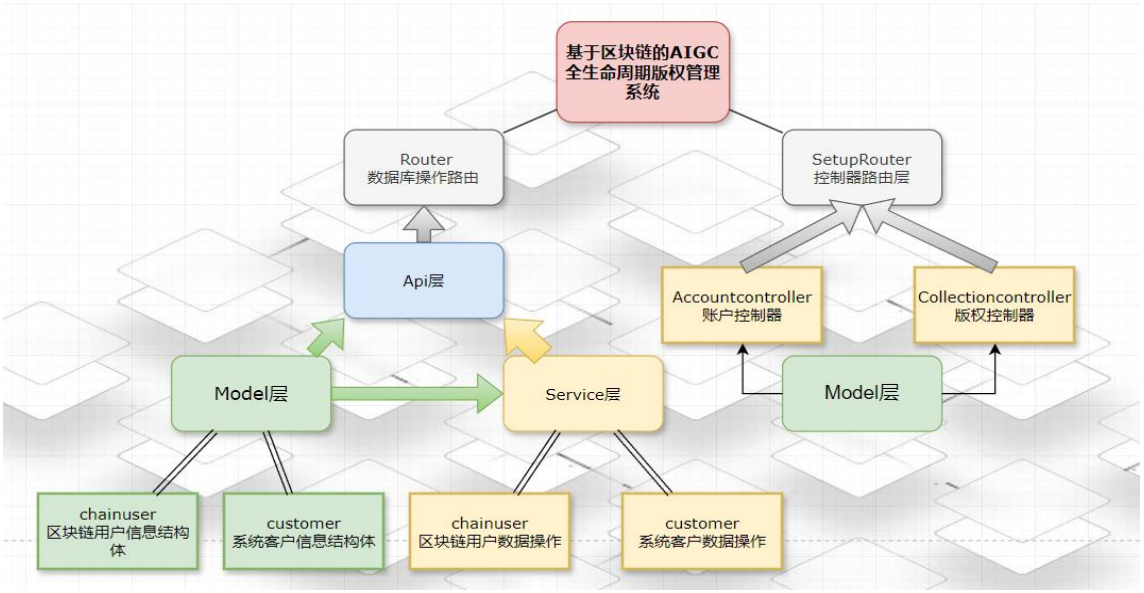


图 4-4 项目后端结构示意图

通过路由层（router）、服务层（service）以及接口层（api）通过分层方式构建整体与数据库互联架构；通过封装调用合约函数形成 controller 模块并添加路由的方式，实现前端操作与链上操作的协同。分层与**模块化**的思想使得项目后端避免了硬编码，后期增添功能变得十分便利，可靠维护变得更加容易的同时可扩展性优异，为未来适配更多服务打下基础。

根据以上方法，实现了前后端交互，信息上链存储等功能。前端展示链上可视内容如图 4-5 所示。



图 4-5 链上版权水印信息查询



图 4-6 版权转让界面

可以看到，系统很好的完成了系统生成内容特征的 SimHash 值的上链存储内容，对生成内容添加唯一标识与认证。用户最终可得到生成内容、并且生成内容相关信息透明可查询。通过链上存证明确生成内容的来源、生成时间和使用权限。同时，用户可通过 *QueryAIGC(queryType, queryValue)* 查询生成内容的相关信息，实现对内容真实性和合法性的验证。

我们对系统的性能和扩展性进行了详细测试，包括 AIGC-CM 与 ERC-3525 和 EIP-712 在不同操作请求率下的操作响应时延以及不同数据集大小下的链式结构和 HSD 的操作响应时延比较，结果如图 4-7 所示，可以看到在创建操作请求速率方面，AIGC-CM 比 ERC-3525 和 EIP-712 具有显著的优势，特别是在高操作请求速率下，而如图 4-8 所示，HSD 相比链式结构，在初始化、创作和查询的 OPT 平均降低分别为

39.21%、28.3%和 53.7%，对于查询和验证等简单操作，节点规模扩大 14 倍时，系统平均响应时间增加不到两倍。

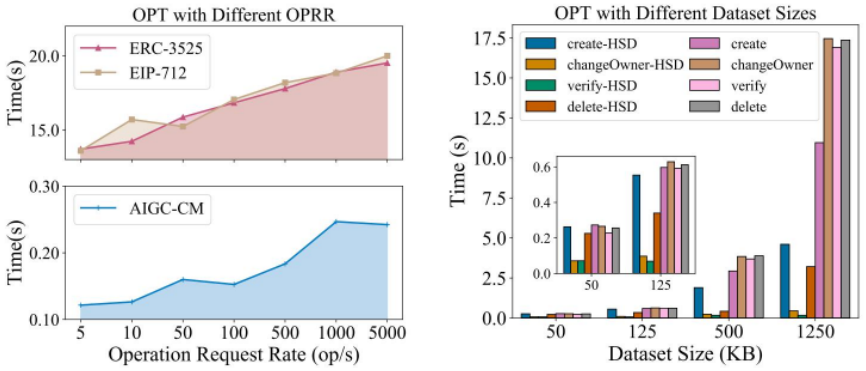


图 4-7 不同操作请求率下的操作响应时延图

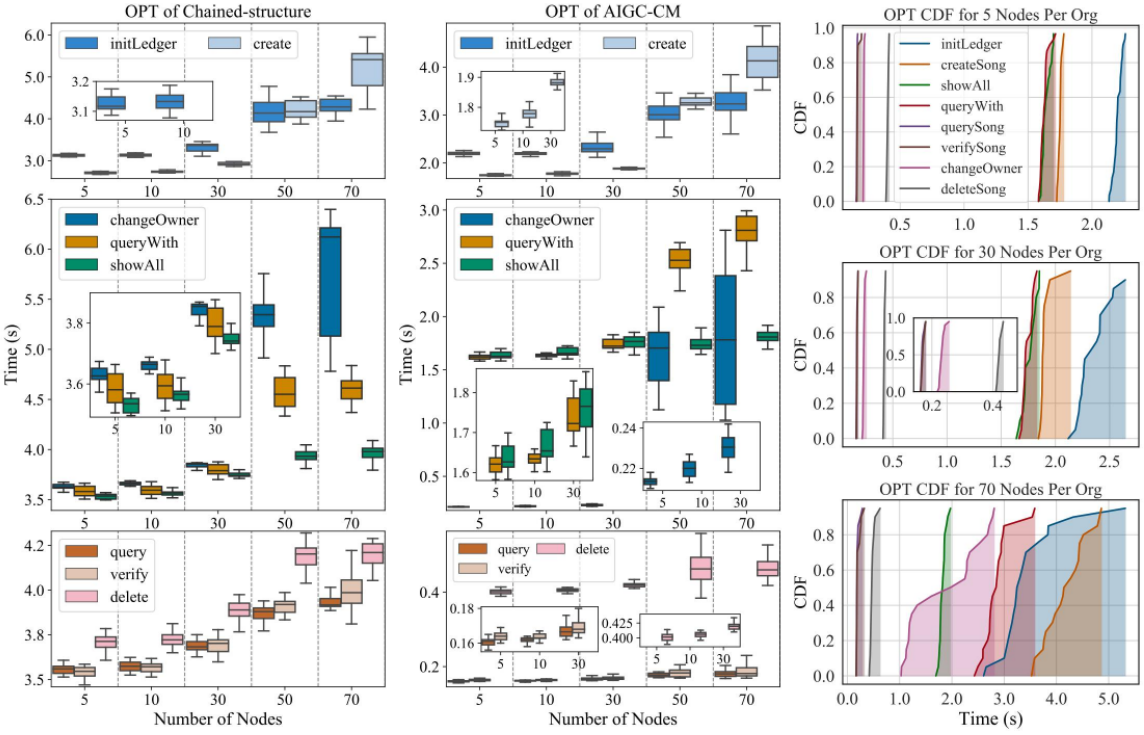


图 4-8 HSD 与链式结构操作响应时延对比图

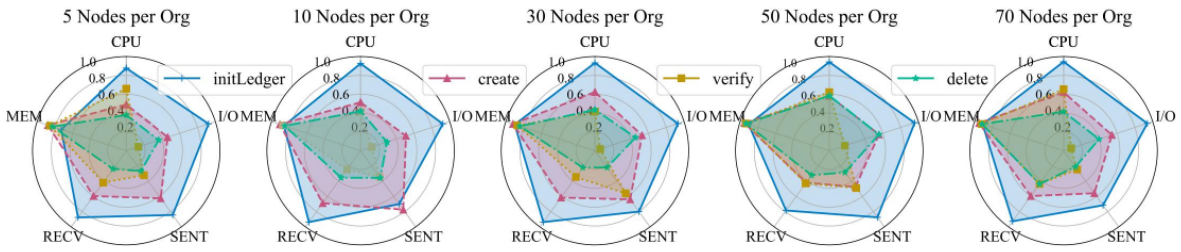


图 4-9 不同节点规模下操作资源利用率图

如图 4-9 所示，我们测试了不同节点规模下不同操作的资源利用情况，总体而言，AIGC-CM 不会对节点造成显著的资源负担，资源利用率波动率（最大比最小的增幅）小于 35%，内存占用率小于 8%。

五、 特色与创新

本项目结合区块链技术与 AIGC 模型生成的独特需求，提供了高效、可扩展的 AIGC 版权管理框架，具有如下特色和创新点：

（1）**精准确权，自动管理。**基于区块链的 MCU（Metadata Copyright Unit）概念，通过特征哈希与结构化元数据实现作品的精确版权认证，自动完成作品的链上自描述与确权，确保生成内容版权的不可篡改性。

（2）**高效验证，智能溯源。**通过创新的 Hysteresis Signature DAG (HSD)认证机制，实现对版权元单元的高效追溯与安全验证，可在复杂的分布式协作环境中快速定位作品来源，满足 AIGC 版权保护的实时性需求。

（3）**安全分析，确保可信。**利用 UC 框架对 AIGC-CM 进行形式化安全分析，确保系统在分布式协作和恶意攻击情况下的安全性，保障版权管理全流程的可信可靠。

（4）**轻量部署，性能卓越。**在 Fabric 2.4 平台上完成 AIGC-CM 的编码部署，经过 140 个云节点的性能测试，实验结果显示在扩展 14 倍节点规模的情况下，操作时间增加不到 2 倍，展示了极高的扩展性与实时响应性能，适用于大规模版权管理场景。

六、 未来展望

本项目以区块链技术为核心，结合生成式 AI 的独特需求，提供了高效、可扩展的 AIGC 版权管理解决方案。未来，团队将继续提升系统的性能与应用广度，推动数字内容产业的健康发展。团队计划结合跨链技术，增强系统在跨域多链环境中的互操作性，确保 AIGC 内容在不同区块链网络之间的版权管理和确权都具有高度一致性。同时，引入隐私计算技术以在多方协作的版权保护场景中保证数据的隐私和安全。探索结合分布式存储与联邦学习技术，在保障数据安全的前提下，进行智能化版权评估与追溯。通过联邦学习，可以实现多方参与下对版权管理数据的联合建模，进一步提高版权验证与追溯的智能化水平，提升整体系统的自适应能力和效率。

团队将优先在 AIGC 创作平台、数字版权认证机构以及版权仲裁中心进行试点部署，通过结合实际使用场景，验证并优化系统的各项功能与性能。未来还计划与内容创作社区和平台合作，逐步推广链上版权确权与管理服务，建立透明、可信的 AIGC 内容生态。最终目标是构建一个开放、共享的 AIGC 内容版权保护生态系统，提供全流程、智能化的版权管理服务，切实保障创作者和用户的版权权益，激发内容创作的活力与创新，推动 AIGC 产业生态的蓬勃发展。

七、 参考文献

[1] Xue X, Shang G, Ma Z, et al. DataSafe: Copyright Protection with PUF Watermarking and Blockchain Tracing[C]. In Proc. of the BWTAC. Springer, 2024: 256-268.

[2] García R, Cediel A, Teixidó M, et al. Semantics and non-fungible tokens for copyright management on the metaverse and beyond[J]. ACM Transactions on Multimedia Computing, Communications and Applications, 2024, 20(7): 1-20.

[3] Liu Y, Du H, Niyato D, et al. Blockchain-empowered lifecycle management for AI-generated content products in edge networks[J]. IEEE Wireless Communications, 2024, 31(3): 286-294.

[4] Lin Y, Du H, Niyato D, et al. Blockchain-aided secure semantic communication for AI-generated content in metaverse[J]. IEEE Open Journal of the Computer Society, 2023, 4: 72-83.

[5] Lin Y, Gao Z, Du H, et al. Blockchain-based Efficient and Trustworthy AIGC Services in Metaverse[J]. IEEE Transactions on Services Computing, 2024, 17(5): 2067-2079.

[6] He X, Xu Q, Lyu L, et al. Protecting intellectual property of language generation apis with lexical watermark[C]. In Proc. of the AAAI. 2022, 36(10): 10758-10766.

[7] Wang B, Jiawei S, Wang W, et al. Image copyright protection based on blockchain and zero-watermark[J]. IEEE Transactions on Network Science and Engineering, 2022, 9(4): 2188-2199.

[8] Benet J. IPFS-content addressed, versioned, p2p file system[J]. arXiv preprint arXiv:1407.3561, 2014.

[9] Wu J, Gan W, Chen Z, et al. AI-Generated Content (AIGC): A survey[J]. arXiv preprint arXiv:2304.06632, 2023.

[10] Natgunanathan I, Praitheeshan P, Gao L, et al. Blockchain-based audio watermarking technique for multimedia copyright protection in distribution networks[J]. ACM Transactions on Multimedia Computing, Communications, and Applications, 2022, 18(3): 1-23.

[11] Peng W, Yi J, Wu F, et al. Are you copying my model? protecting the copyright of large language models for eaas via backdoor watermark[J]. arXiv preprint arXiv: arXiv:2305.10036, 2023.

[12] Chen C, Wu Z, Lai Y, et al. Challenges and remedies to privacy and security in AIGC: Exploring the potential of privacy computing, blockchain, and beyond[J]. arXiv preprint arXiv:2306.00419, 2023.

[13] O. AI. (2022) AIGC Chain. Accessed on January 30, 2024. [Online]. Available: https://www.aigcchain.io/about

[14] Ma L, Liu X, Li Y, et al. GFBE: A Generalized and Fine-Grained Blockchain Evaluation Framework[J]. IEEE Transactions on Computers, 2024, 73(3): 942-955.

[15] Ma L, Hu D, Liu X, et al. AIGC-CM: An Efficient and Scalable Blockchain Solution for AIGC Copyright Management[C]. In Proc. of the IEEE INFOCOM, 2025.

八、 附件

1. 文生图并链上存储版权信息（对应文生图页面）

Step1：文生图，调后端 runsd 调用本地路径模型生成图片，并且保存至指定路径；

Step2：前端反馈图片，选择信息是否上链；

Step3：点击确定，信息上链，调后端 sendToBlock 生成图片后向链上传输信息 sendToBlock；

函数名	功能描述
ImgAddWatermark	添加水印
imageToMatrix	图片转换为等大矩阵
rgbaMatrixToFloat64Matrix	图片转换为 float64 类型

Step4：sendPostRequest 根据 url 调用链上 insert 方法，发送请求，内容上链前端可视返回信息。

名称	模块	备注
链上编号	collection_id	可以根据此 id 至查询界面查询图片详细信息
哈希内容	tx	交易哈希

2. 依据链上返回 id 查询图片信息（对应查询页面）

调后端 selectFromChains，封装更底层 select_from_chains，调链上 transfer 接口交互，前端输入参数 collection_id，返回信息。

名称	模块	说明
链上编号	collection_id	用于查询图片详细信息的唯一标识
图片名称	collection_name	图片的名称或标题
所有者用户名	owner_id	当前图片所有者的用户名
认证时间	certificate_time	图片认证的时间戳
认证机构	certificate_organization	进行认证的机构名称
语义	collection	图片的语义描述或标签

删除区块：调后端 deletefromchains 封装底层 delete_from_chains，调链上 delete 接口交互，删除链上相关信息。

3. HSD 存储版权信息与查询

我们的项目采用了有向图结构进行版权信息的存储，版权信息以节点的方式存储在 HSD 上，支持对存储着版权信息的节点进行添加、删除、更改和查询操作，保证高效率的同时保证了功能的全面性和操作的灵活性。更进一步的，在结构上，每一个节点通过相似度将子节点划分为 10 组，在算法上，我们设计了确定性的插入，删除，更改，查询算法，以达到在保证正确性的同时，尽可能的保证高插入效率。

节点插入：节点插入采用递归插入模式。具体来说，首先，根据插入节点(x)和当前父节点(fa)的$SimHash$，计算哈希相似度$(SimHash_{fa})$，而后根据$SimHash_{fa}$确定划分到的组，找到组内与x相似度最大的几个节点，并计算相似度$(SimHash_{son})$，若$SimHash_{fa} > SimHash_{son}$，则直接$x$将插入到对应子结点组，否则递归地将插入子树中。插入节点后，向上更新相关信息。

节点查询：类似于节点插入，节点查询采用递归查询模式。首先，根据查询节点(x)和当前父节点(fa)的$SimHash$，计算哈希相似度，若完全一致，则找到节点，否则根据$SimHash_{fa}$确定划分到的组，找到组内与(q)相似度最大的几个节点，并计算相似度$SimHash_{son}$，然后进行比较，若$SimHash_{fa} > SimHash_{son}$，则在对应子结点组中寻找，否则递归地在子树中查询，若最后返回空节点，则认为 HSD 中没有这个节点。

节点删除：节点删除，我们先查找到对应的节点，而后进行删除和删除后修正操作。具体来说，我们首先调用节点查询功能，定位到要删除的节点(d)。如果查找失败，进行报错处理，否则进行删除操作：如果该节点没有儿子节点，直接执行删除操作，否则，将该节点的儿子$(sons)$临时存储，删除节点(d)，将$(sons)$中的节点全部重新执行节点插入操作。删除之后，向上更新相关信息。

节点修改：节点修改，我们先查找到对应的节点，而后进行修改和修改后的工作。具体来说，我们首先 调用节点查询功能，定位到要进行修改的节点u。如果查找失败，进行报错处理，否则进行修改操作：进行信息修改，修改完成后，向上逐步更新相关信息。

总之，我们原创性地提出有向无环图结构，实现了对版权信息简明而清晰的存储以及正确且灵活的操作。

4. 链下计算与链上存储交互认证

由于链上开销限额问题，我们提出了“链下计算与链上存储交互认证”模式，具体来说，链上存储关键信息，链下存储具体信息，链上链下同时查询，查询结果交互认证，从而在保证正确性的同时，降低了链上开销。

一方面，链上存储如下信息：

参数	类型	说明
user_id	string	用户 id
user_money	*big.Int	用户账户余额
user_name	string	用户名称
user_icon	string	用户头像地址

另一方面，链下存储如下信息：

参数	类型	说明
Matrix	uint[][]	矩阵
Make	string	创作者
Record	string	记录
Owner	string	所有者
Style	string	实体类型

具体的运行逻辑是，根据需要在链上和链下同时进行信息定位，然后将定位到的信息对比验证，如果验证不匹配，那么我们认为链下信息遭到了恶意破坏并进入错误处理，否则进行后续操作：首先，在链下生成一对公私钥，用于签名和验证。在程序启动时调用初始化函数生成密钥对。然后，在链下对数据进行签名，确保数据的完整性和不可篡改性。签名函数对输入数据进行哈希计算，并使用私钥对哈希值进行签名，返回签名结果。之后，将签名后的数据和其他相关信息发送到区块链上。插入函数连接到区块链客户端，加载智能合约实例，并将数据插入到区块链中。最后在链上或链下验证数据的签名，确保数据的真实性。验证函数对输入数据进行哈希计算，并使用公钥验证签名的有效性，返回验证结果。

5. 链上存储合约设计

对于链上的数据，采用模拟数据库的方式进行存储。支持选择（select），插入（insert），更改（update），删除（delete）四个功能，由于我们采用了链下计算与链上存储交互认证的创新方法，这使得我们的链上存储十分轻量化，我们设计出的模拟数据库也可以准确且高效的实现目标功能。

具体来说，四个功能的实现方式如下：

插入记录：智能合约通过工厂接口访问特定的数据表，并创建一个新的数据条目。然后，它将接收到的数据填充到这个条目中，并将其插入到表中。这个过程会记录操作影响的行数，并触发一个事件来标记 插入操作的结果。

更新记录：智能合约首先访问数据表，然后创建一个新的数据条目来存储更新后的值。接着设置一个条件来指定哪些记录需要更新。使用这个条件，合约更新符合条件的记录，并记录更新操作影响的行 数，同时触发一个事件来记录这次更新的结果。

删除记录：在删除功能中，智能合约打开数据表并设置一个条件来定义哪些记录将被删除。然后，它执行删除操作，并记录删除操作影响的行数，同时触发一个事件来记录这次删除的结果。

查询记录：智能合约打开数据表并执行查询操作，检索出所有符合条件的记录。它遍历查询结果，提取每条记录的相关数据，并返回这些数据的集合，这样用户就可以获取到他们请求的信息。

这些功能共同实现了智能合约中对数据的基本管理，确保了数据的安全性和操作的透明性，四个基础而简介的功能全面而灵活。

6. 链端接口函数

接口函数是链与后端交互的部分，具体流程如下：

首先，将需要发送的数据准备好，并将其序列化为 JSON 格式。这一步确保数据在传输过程中能够被正确解析。接下来，创建一个 HTTP POST 请求，指定目标 URL，并将序列化后的 JSON 数据作为请求体。这一步设置了请求的基本信息，包括请求方法、目标地址和请求内容。在创建好的请求中，设置请求头的 Content-Type 为 application/json，以告知服务器请求体的内容类型是 JSON。这一步确保服务器能够正确解析请求体中的数据。之后，发送请求，使用 HTTP 客户端发送创建好的请求，并等待服务器的响应。这一步将请求发送到指定的服务器，并接收服务器返回的响应。接收到服务器的响应后，检查响应的状态码和内容。如果请求成功，处理响应中的数据；如果请求失败，记录错误信息。这一步确保能够正确处理服务器返回的数据或错误信息。

通过上述步骤，可以实现与后端的交互。首先准备并序列化数据，然后创建并配置 HTTP 请求，发送请求并接收响应，最后处理响应中的数据或错误信息。这个流程确保数据能够在客户端和服务器之间正确传输和解析。